FPT UNIVERSITY

Capstone Project Report

Report 2 – Project Management Plan

– Can Tho, October 2026 –

Table of Contents

I. Record of Changes . . . 3

II. Project Management Plan . . . 4

1. Overview . . . 4

1.1 Scope & Estimation . . . 4

1.2 Project Objectives . . . 14

1.2.1 Key Performance Objectives . . . 15

1.2.2 Testing Strategy & Quality Gates . . . 16

1.3 Project Risks . . . 16

2. Management Approach . . . 21

2.1 Project Process . . . 22

2.2 Quality Management . . . 24

2.3 Training Plan . . . 27

3. Project Deliverables . . . 29

4. Responsibility Assignments . . . 30

4.1 Team & Structure . . . 31

4.2 Role & Responsibilities . . . 31

4.3 Responsibility Assignment Matrix . . . 33

5. Project Communications . . . 35

6. Configuration Management . . . 36

6.1 Document Management . . . 36

6.2 Source Code Management . . . 37

6.3 Tools & Infrastructures . . . 38

I. Record of Changes

Date	A* M, D	In charge	Change Description

**A - Added M - Modified D - Deleted*

II. Project Management Plan

1. Overview

1.1 Scope & Estimation

The work breakdown below is the basis for sprint planning. Each row is a SMAR (Specific, Measurable, Achievable, Relevant) item that can be demonstrated at the end of the sprint in which it is scheduled, and the identifier in the second column is the reference used by the task board, the review checklist, and the test cases. Complexity is rated Low, Medium or High according to technical uncertainty and integration risk rather than to effort alone; effort is estimated in man-days and rounded to whole days. Module header rows carry no estimate because their effort is already distributed across the items beneath them.

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
1		**Authentication**		–	–
2	1.1	Register with email one-time-code verification		Medium	3
3	1.2	Google OAuth sign-in and account linking		Medium	3
4	1.3	Login and logout with JWT issuance	Sprint 1	Low	2
5	1.4	Refresh token rotation and persisted sessions		Medium	3
6	1.5	Password reset through a one-time code		Low	2
7	1.6	Password change from account settings		Low	1
8	1.7	Multi-role membership, join role and role switching		Medium	3
9	1.8	Account profile with avatar and cover upload		Medium	3
10		**Authorization and Administration**		–	–
11	1.9	Permission model bound to method, path and module		Medium	3
12	1.10	Permission guard and public route decorator	Sprint 1	Medium	3

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
13	1.11	Administrative role management		Medium	2
14	1.12	Administrative permission assignment		Medium	2
15	1.13	Administrative user creation and update		Medium	2
16	1.14	Account ban and unban		Low	1
17	1.15	Session listing, detail and revocation		Medium	2
18		**Profile and Portfolio**		–	–
19	1.16	Freelancer profile with education and certifications		Medium	3
20	1.17	Work experience entries with dates and description		Low	2
21	1.18	Client profile with company information	Sprint 1	Low	1
22	1.19	Skills with proficiency level and suggestions		Medium	2
23	1.20	Portfolio items with technologies and cover image		Medium	3
24	1.21	Social link management		Low	1
25	1.22	Profile completion and strength scoring		Medium	2
26	1.23	Profile revision submission and approval		Medium	3
27		**Curriculum Vitae Handling**		–	–
28	1.24	CV upload and secure storage		Medium	2
29	1.25	CV preview and download	Sprint 1	Low	1
30	1.26	CV replacement and deletion		Low	1
31		**Job Marketplace**		–	–
32	1.27	Job creation with budget, deadline, skills and categories		Medium	3
33	1.28	Job editing and deletion with ownership validation		Medium	2
34	1.29	Job status lifecycle	Sprint 1	Medium	2
35	1.30	Client job list and detail		Medium	2
36	1.31	Public job search with keyword matching		Medium	2

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
37	1.32	Public job filters by category, skill and budget range		Medium	3
38	1.33	Job detail page with contextual actions		Medium	2
39	1.34	Job category browsing		Low	1
40	1.35	Unique, stable slug generated for every published job		Low	2
41		**Bookmarks and Saved Search**		–	–
42	1.36	Job bookmark save, remove and status	Sprint 1	Low	1
43	1.37	Bookmarked job list		Low	1
44	1.38	Saved search creation and listing		Medium	2
45	1.39	Saved search update and deletion		Medium	2
46		**Community Forum Foundation**		–	–
47	1.40	Forum category listing		Low	1
48	1.41	Forum post creation with rich text and tags	Sprint 1	Medium	3
49	1.42	Forum post listing, search and detail		Medium	2
50	1.43	Comments, likes and reporting		Medium	3
51	1.44	Administrative approval of pending posts		Low	1
52		**Identity Verification Capture**		–	–
53	1.45	Identity document upload with front and back images	Sprint 1	Medium	3
54	1.46	Selfie capture and liveness frame submission		Medium	3
55	1.47	AI verification request, timeout and result handling		High	5
56	1.48	Verification status display		Medium	2
57		**Proposal Management**		–	–
58	2.1	Proposal creation with bid, duration and cover letter		Medium	2
59	2.2	Proposal attachments		Low	1
60	2.3	Save proposal as draft		Medium	2
61	2.4	Proposal editing	Sprint 2	Medium	2
62	2.5	Proposal withdrawal		Low	1

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
63	2.6	Proposal expiry handling		Medium	2
64	2.7	Freelancer proposal list and filters		Low	2
65	2.8	Client proposal list for a job		Low	2
66	2.9	Proposal detail with applicant profile		Low	1
67		**Hiring Decision**		–	–
68	2.10	Accept proposal and create contract	Sprint 2	Medium	3
69	2.11	Reject proposal		Low	1
70	2.12	Position count enforcement for multiple hiring		Medium	2
71		**Contract Management**		–	–
72	2.13	Contract generation from an accepted proposal		Medium	3
73	2.14	Contract list and detail for both parties	Sprint 2	Medium	2
74	2.15	Dual contract signing		Medium	3
75	2.16	Platform fee recording and contract total		Medium	2
76	2.17	Contract completion		Medium	2
77	2.18	Contract cancellation		Medium	2
78		**Milestone Management**		–	–
79	2.19	Milestone list and detail		Low	1
80	2.20	Milestone creation		Medium	2
81	2.21	Milestone editing		Medium	2
82	2.22	Milestone deletion and start-in-progress transition	Sprint 2	Low	1
83	2.23	Submission with message and external links		Medium	2
84	2.24	Submission file upload and removal		Medium	2
85	2.25	Change request with message and due date		Medium	2
86	2.26	Submission approval and completion trigger		Medium	3
87		**Milestone Files**		–	–
88	2.27	Milestone file listing	Sprint 2	Low	1

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
89	2.28	Milestone file upload and type validation		Medium	2
90	2.29	Milestone file deletion		Low	1
91		**Milestone Progress**		–	–
92	2.30	Start a milestone (pending to in progress)	Sprint 2	Low	1
93	2.31	Milestone state display in the contract view		Low	1
94		**Contract Shared Files**		–	–
95	2.32	Shared file upload	Sprint 2	Medium	2
96	2.33	Shared file listing and deletion		Low	1
97		**Review and Rating**		–	–
98	2.34	Contract review with overall rating and breakdown		Medium	3
99	2.35	Review editing and deletion	Sprint 2	Low	1
100	2.36	Contract review listing		Low	1
101	2.37	Review detail		Low	1
102	2.38	Review response creation		Low	1
103	2.39	Review response update and deletion		Low	1
104		**Real-time Messaging**		–	–
105	3.1	Conversation model and conversation list		Medium	2
106	3.2	Starting a conversation		Medium	2
107	3.3	Socket authentication and gateway setup	Sprint 3	Medium	3
108	3.4	Sending text messages		Medium	2
109	3.5	File attachment messages		Medium	2
110	3.6	Read receipts		Low	1
111	3.7	Typing indicator		Low	1
112	3.8	Message deletion		Low	1
113	3.9	Conversation pinning, hiding and unread count		Medium	2
114		**Notification Centre**		–	–
115	3.10	Notification creation from domain events		Medium	2
116	3.11	Notification list with typed entries	Sprint 3	Medium	2

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
117	3.12	Unread count and badge		Low	1
118	3.13	Mark as read and mark all as read		Low	1
119	3.14	Notification deletion		Low	1
120		**Saved Search Refinement**		–	–
121	3.15	Saved search filters by category and budget	Sprint 3	Low	1
122	3.16	Saved search order and pagination		Low	1
123	3.17	Saved search naming and rename		Low	1
124	3.18	Saved search application to the job search screen		Low	1
125		**Session Management**		–	–
126	3.19	Active session list with device and IP address	Sprint 3	Medium	2
127	3.20	Session detail		Low	1
128	3.21	Session revocation		Low	1
129		**Permission Administration**		–	–
130	3.22	Permission creation		Medium	2
131	3.23	Permission listing and editing	Sprint 3	Low	1
132	3.24	Permission deletion and role binding review		Low	1
133		**User Administration**		–	–
134	3.25	Administrative user list with search and filters		Medium	2
135	3.26	Administrative user detail		Low	1
136	3.27	Administrative user creation	Sprint 3	Medium	2
137	3.28	Administrative user update		Medium	2
138	3.29	Ban and unban accounts		Low	1
139	3.30	Administrative client profile editing		Low	1
140	3.31	Administrative freelancer profile editing		Low	1
141	3.32	Skill catalogue lookup for administrative forms		Low	1
142		**Profile Revision Review**		–	–
143	3.33	Revision queue with pending and approved filters	Sprint 3	Medium	2

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
144	3.34	Comparison of proposed and current profile data		Medium	2
145	3.35	Approve or reject a revision		Low	1
146		**Identity Verification Review**		–	–
147	3.36	Verification review queue		Medium	2
148	3.37	Document viewer with access control	Sprint 3	Low	1
149	3.38	Verification decision and badge publication		Medium	2
150		**Forum Categories**		–	–
151	3.39	Forum category creation		Low	1
152	3.40	Forum category listing and search	Sprint 3	Low	1
153	3.41	Forum category editing		Low	1
154	3.42	Forum category deletion		Low	1
155		**Forum Moderation**		–	–
156	3.43	AI moderation payload ingestion		Medium	2
157	3.44	Moderation queue with risk filters		Medium	2
158	3.45	Approve and reject actions from the queue	Sprint 3	Low	1
159	3.46	Report handling and resolution		Low	1
160	3.47	Forum trash and restore		Low	1
161		**Stripe Connect Onboarding**		–	–
162	4.1	Express account onboarding link		Medium	3
163	4.2	Onboarding status synchronisation	Sprint 4	Medium	2
164	4.3	Saved payment method listing		Low	1
165	4.4	Customer portal session		Low	1
166		**Milestone Funding**		–	–
167	4.5	Platform fee PaymentIntent		Medium	3
168	4.6	Milestone funding PaymentIntent		Medium	3
169	4.7	Hosted Checkout Session for funding	Sprint 4	Medium	2
170	4.8	Funding with a saved card		Medium	2
171	4.9	Funding state synchronisation		Medium	2
172		**Release, Refund and Ledger**		–	–
173	4.10	Milestone release as a Stripe transfer	Sprint 4	High	4
174	4.11	Refund processing		Medium	3

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
175	4.12	Dispute settlement with split payouts		Medium	3
176	4.13	Transaction ledger with type and status		Medium	2
177		**Payment Webhooks**		–	–
178	4.14	Webhook endpoint with signature verification	Sprint 4	Medium	3
179	4.15	PaymentIntent event handling		Low	1
180	4.16	Checkout Session event handling		Low	1
181		**Dispute Management**		–	–
182	4.17	Dispute creation with claim evidence		Medium	3
183	4.18	Arbitration fee payment intent		Medium	2
184	4.19	Fee and response deadline handling	Sprint 4	Medium	2
185	4.20	Dispute response with evidence		Medium	2
186	4.21	Dispute list and detail for participants		Low	1
187	4.22	Decision review request		Medium	2
188	4.23	Administrative decision with split and reasoning		Medium	3
189	4.24	Final decision and dispute closure		Low	1
190		**Advertisement Banners**		–	–
191	4.25	Banner creation		Low	1
192	4.26	Banner listing		Low	1
193	4.27	Banner editing	Sprint 4	Low	1
194	4.28	Banner deletion and activation window		Low	1
195	4.29	Public banner display by placement position		Low	1
196		**Skill Catalogue**		–	–
197	4.30	Skill creation		Low	1
198	4.31	Skill listing and search		Low	1
199	4.32	Skill editing	Sprint 4	Low	1
200	4.33	Skill soft deletion		Low	1
201	4.34	Skill restore		Low	1
202		**Job Categories**		–	–

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
203	4.35	Category creation	Sprint 4	Low	1
204	4.36	Category listing and search		Low	1
205	4.37	Category editing		Low	1
206	4.38	Category soft deletion		Low	1
207	4.39	Category restore and status toggle		Low	1
208		**AI Curriculum Vitae Analysis**	Sprint 5	–	–
209	5.1	AI service client with timeout and retry policy		Medium	3
210	5.2	Skill suggestion parsing and validation		Medium	2
211	5.3	Suggested skill review interface		Medium	2
212	5.4	Graceful degradation when the AI service is unavailable		Low	1
213		**Expert Directory**	Sprint 5	–	–
214	5.5	Expert profile editing with revision approval		Medium	2
215	5.6	Public expert list with filters		Low	1
216	5.7	Public expert detail		Low	1
217	5.8	Administrative expert profile editing		Medium	2
218		**Social Features**	Sprint 5	–	–
219	5.9	Follow and unfollow a freelancer		Low	1
220	5.10	Follower and following lists		Low	1
221	5.11	Freelancer discovery with filters		Medium	2
222	5.12	Favourite freelancer toggle		Low	1
223	5.13	Favourite freelancer list		Low	1
224	5.14	Public freelancer profile page with skills, portfolio and experience		Low	1
225		**Trust Indicators**	Sprint 5	–	–
226	5.15	Verified badge rendering on public profiles		Low	1
227	5.16	Availability and online status display		Low	1
228	5.17	Profile completion indicator in navigation		Low	1

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
229	5.18	Verification status messaging and error states		Low	1
230		**Localisation and Presentation**		–	–
231	5.19	Bilingual message catalogue for all screens		Medium	4
232	5.20	Locale switcher with cookie persistence	Sprint 5	Low	1
233	5.21	Light and dark theme switcher		Low	1
234	5.22	Responsive layout adjustments		Medium	2
235	5.23	Public banner and announcement placement		Low	1
236		**Administrative Dashboards**		–	–
237	5.24	Identity verification dashboard		Medium	2
238	5.25	Profile revision dashboard	Sprint 5	Medium	2
239	5.26	Moderation dashboard		Medium	2
240	5.27	Report detail view		Low	1
241		**Quality Hardening**		–	–
242	5.28	Shared validation schemas for all endpoints		Medium	2
243	5.29	Response envelope and global exception filter		Medium	2
244	5.30	Guard and shared-service test coverage	Sprint 5	Medium	2
245	5.31	Repository and service unit tests		Medium	2
246	5.32	API bootstrap end-to-end test		Medium	2
247	5.33	File upload validation and size limits		Low	1
248	5.34	Environment configuration validation		Low	1
249	5.35	Docker Compose for the API and web client		Medium	2
250	5.36	Database migration and seed scripts		Low	1
251	5.37	Demo accounts and sample data for acceptance testing		Low	1
252		**Documentation and Acceptance**		–	–

No	#	SMAR Item	Iteration Code	Complexity	Est. Effort (man days)
253	5.38	API reference and onboarding guide	Sprint 5	Medium	2
254	5.39	Deployment and environment guide		Low	1
255	5.40	User acceptance testing and defect fixes		Medium	3
256	5.41	Final stabilisation and release preparation		High	4

Table 2 aggregates the same rows by sprint. The five sprints cover sixteen calendar weeks: the first sprint runs for four weeks to absorb project setup and the foundational identity and job modules, while the remaining four run for three weeks each.

Table 2: Estimation Summary by Sprint

Sprint	Weeks	Modules	SMAR Items	Est. Effort (man days)
Sprint 1	1–4	8	48	104
Sprint 2	5–7	8	39	67
Sprint 3	8–10	10	47	67
Sprint 4	11–13	8	39	67
Sprint 5	14–16	8	41	67
Total	**16 weeks**	**42**	**214**	**372**

The four three-week sprints share an identical effort envelope of 67 man-days, which keeps the team's velocity assumption constant after the first sprint and makes an overrun immediately visible: a sprint that consumes more than 67 man-days is a signal to remove scope rather than to extend the sprint. Sprint 4 carries the highest integration risk because payments and disputes are scheduled together, and Sprint 5 deliberately reserves effort for hardening and acceptance rather than for new functional scope.

1.2 Project Objectives

The **Frevia** project (project code SEP490_04) develops a web-based freelance marketplace that connects Clients and Freelancers in software, design, content, and consulting work. A Client publishes a job with a fixed-price budget, a deadline, required skills, categories, and the number of positions to fill; Freelancers submit proposals, either as drafts or as submitted applications with attachments and an expiry date; the Client compares proposals and hires a

candidate, which generates a contract that both parties must sign. The contract carries a total amount and a fixed platform fee recorded on the contract, and the work is divided into milestones that are funded into escrow, submitted with files and links, optionally returned for changes with a due date, and, after approval, released as a Stripe payout. Completed contracts can be reviewed with an overall rating and a category breakdown, and each review can receive a response.

Beyond the core transaction, the platform provides electronic identity verification with an AI document and liveness pipeline, an administrative review queue for identity documents and profile revisions, a milestone-level dispute procedure with an arbitration fee, evidence, deadlines, an administrative decision, and a review step, one-to-one real-time messaging over an authenticated WebSocket, a persistent notification record, a community forum with categories, comments, likes, reporting, and AI-assisted moderation, saved searches and job bookmarks, an expert directory, advertisement banners, and a data-driven permission model that lets an administrator change what each role may do without a redeployment. Artificial intelligence is applied only to curriculum vitae skill extraction, identity verification, and forum content moderation; no recommendation, ranking, or automatic matching engine is part of the scope.

Based on the established backlog of **214 SMAR items** distributed across five sprints, the total estimated effort is **372 man-days**, as detailed in the *Scope and Estimation* table and summarised in Table 2.

1.2.1 Key Performance Objectives

- **Timeliness (%):** Deliver all sprints and milestones on schedule.
  Target: 95% of the tasks (i.e., 203 out of 214) must be completed within the sprint in which they are planned. Delayed tasks shall not exceed 5% of the total.

- **Allocated Effort (man-days):** Optimise development resources, aligning effort with task complexity, rated low, medium, or high.
  Target: Actual total effort shall not exceed $372 \pm 5\%$ (max. 391 man-days). High-complexity tasks (the identity verification pipeline, Stripe escrow funding and webhooks, dispute arbitration, and the real-time messaging gateway) are assigned to experienced team members and scheduled early in their sprint.

- **Defect Distribution:** Maintain high software quality throughout development, testing, and acceptance.
  Target:

  - No critical defects remain open at the acceptance milestone.
  - 95% of high-priority defects fixed within 48 hours of detection.
  - Medium-priority defects not exceeding 5% of the total system defects.

1.2.2 Testing Strategy & Quality Gates

All 214 items must pass through the following testing stages. Table 3 summarises the test coverage targets and the defect resolution key performance indicators.

Table 3: Testing Stages and Quality Targets

#	Testing Stage	Test Coverage (%)	No. of Defects	% of Defects Solved
1	Reviewing (code review and design review)	100	0	100
2	Unit Test	80	0	100
3	Integration Test	100	0	100
4	System Test	100	0	100
5	Acceptance Test	95	0	100

- Reviewing includes peer review and static analysis with ESLint for both the NestJS API and the Next.js web client, together with explicit review of every Stripe call site and webhook handler.
- Unit Test requires at least 80% coverage for critical modules: authentication, access-token validation, job and proposal handling, contract and milestone state transitions, payment services, and the dispute service.
- Integration Test verifies the main flow: create job $\rightarrow$ submit proposal $\rightarrow$ accept proposal $\rightarrow$ generate and sign contract $\rightarrow$ create milestone $\rightarrow$ fund the milestone into escrow $\rightarrow$ submit work $\rightarrow$ approve the submission $\rightarrow$ release the payment.
- System Test executes end-to-end scenarios on a staging environment using Stripe test mode with simulated webhook events, a test identity document set for the verification pipeline, and a sample curriculum vitae for skill extraction.
- Acceptance Test is performed with at least 3 Clients and 5 Freelancers, ensuring that 95% of the key business workflows function correctly.

1.3 Project Risks

The following risks have been identified from the project backlog and from the technical characteristics of the system: payment escrow, an external identity verification pipeline, a real-time messaging layer, and a dispute process that produces decisions with financial consequences. Each risk is rated by impact and possibility, and is paired with a response plan that is owned by a named role in Section 4.2.

#	Risk Description	Impact	Possibility	Response Plans
1	**Stripe escrow and webhook failures** (PaymentIntent and Checkout Session funding, transfer release, refund, and duplicate or out-of-order webhook events)	High	High	Develop and test exclusively in Stripe test mode. Verify the signature against the raw request body, make every handler idempotent by keying on the Stripe event identifier, and always reconcile local state from Stripe before writing. Keep an explicit synchronisation endpoint so an operator can repair a stuck record without a database edit.
2	**Stripe Connect onboarding and regional payout limits** (Express account requirements, country availability, and capability restrictions)	High	Medium	Detect onboarding completion from the account status instead of assuming success after the redirect. Block payout-dependent actions until the account is verified, show the reason to the user, and document the supported countries in the limitation section of Report 1.
3	**External AI service unavailability** (curriculum vitae analysis and identity verification both call services outside the monorepo)	High	High	Keep both integrations behind a single service class with an explicit timeout. Return a clear error instead of a silent failure, allow the user to retry, and keep administrative manual review as the fallback for identity verification. Develop the frontend against a stub response so that UI work is never blocked by the AI service.

#	Risk Description	Impact	Possibility	Response Plans
4	**Identity document accuracy and privacy** (sensitive personal documents are uploaded, stored, and reviewed by administrators)	High	Medium	Serve identity documents only through authenticated endpoints with ownership or administrator checks, never through the public static directory. Store only the verification outcome and summary on the profile, restrict the review queue to the administrator role, and define a retention rule for rejected submissions.
5	**Arbitration load and perceived fairness** (an administrator must decide disputes with real money at stake)	Medium	High	Keep the evidence structure symmetric, record the reasoning with every decision, and expose the split amounts to both parties. Publish a target of 48 hours for a first decision, provide the review request as an escalation path, and rotate the deciding administrator when a review is requested.
6	**Scope creep across five sprints** (214 SMAR items and a fixed envelope of 372 man-days)	High	High	Freeze the backlog at sprint planning. Any addition must be exchanged for an item of comparable effort rather than added on top, and the sprint commitment may not exceed 67 man-days. Total effort must stay within $372 \pm 5\%$ (max. 391 man-days).

#	Risk Description	Impact	Possibility	Response Plans
7	**Prisma schema evolution** (milestone payment state, dispute state, and transaction types change during delivery)	Medium	High	Treat migrations as reviewable artefacts, generate them from the schema rather than editing the database by hand, and apply them to a staging database before merging. Keep enums aligned with the state machines described in the design document so that a new status cannot be introduced without updating both the API and the interface.
8	**Monorepo toolchain drift** (Next.js, React, Tailwind, Prisma, and NestJS are on recent major versions)	Medium	Medium	Pin versions in the lockfile, upgrade one dependency at a time on a dedicated branch, and require the full pipeline (lint, type check, unit tests, and a Docker build) to pass before merging. Keep the shared packages built from the same TypeScript version as the applications.
9	**Real-time layer limited to one API instance** (WebSocket rooms are held in memory on a single process)	Medium	Medium	Document the limitation explicitly, avoid designing any feature that requires cross-instance presence, and keep the gateway isolated so that a shared adapter can be introduced later without changing the event contract. Load-test the messaging flow with concurrent clients before acceptance.

#	Risk Description	Impact	Possibility	Response Plans
10	**Third-party service availability** (Google OAuth, SMTP for one-time codes, and Cloudinary for media)	Medium	Medium	Keep email and password authentication available at all times so that a Google outage does not lock users out. Surface delivery failures to the user with a resend option, and keep uploaded documents on the application's own storage path so that a media provider outage cannot block identity verification.
11	**Integration testing bottleneck** (many modules share the contract, milestone, and payment state machines)	Medium	High	Write the end-to-end path first and extend it as modules land: job, proposal, contract, signature, milestone, funding, submission, approval, and release. Keep repository-level tests for state transitions, and run the full suite before every merge rather than only at the end of the sprint.
12	**Team knowledge gap** (only part of the team has prior experience with Stripe, Prisma migrations, or Socket.IO)	High	Medium	Run internal sessions before Sprint 2 (Prisma and NestJS patterns) and before Sprint 4 (Stripe escrow and webhooks). Pair an experienced member with the owner of each high-complexity item, and require a code review on every payment and verification change.

#	Risk Description	Impact	Possibility	Response Plans
13	**Data privacy and retention** (personal data, identity documents, and financial records are stored together)	Medium	Medium	Avoid writing personal data into application logs, restrict document access by role, and define what is retained after a dispute is finalised. Keep the privacy notice and the consent step aligned with what the system actually collects.

2. Management Approach

The Frevia project adopts Scrum as its primary management approach, with a fixed delivery window of sixteen weeks divided into five sprints. Scrum is used because the product is not fully specified at the start: the payment, dispute, and verification flows each depend on decisions that only become clear once an earlier increment exists and has been reviewed. An iterative process allows the team to deliver a working slice of the marketplace in every sprint instead of accumulating unverified work until the end.

Three artefacts govern the work. The Product Backlog holds the 214 SMAR items listed in Section 1.1, each with an identifier, a complexity rating, and an effort estimate in man-days. The Sprint Backlog is the subset committed at Sprint Planning, and it is capped at 67 man-days for Sprints 2 to 5 and at 104 man-days for the four-week first sprint. The Increment is what the team demonstrates at the Sprint Review: a running system in which the committed items are usable end to end, not a set of unfinished branches.

The cadence differs deliberately between the first sprint and the rest. Sprint 1 runs for four weeks because it carries the architectural decisions that everything else depends on: the monorepo layout, the shared validation package, the data model, the authentication flow, and the permission guard. Sprints 2 to 5 run for three weeks each and share an identical effort envelope, which makes velocity comparable across them and makes an overrun visible early rather than at the end of the project. Each sprint ends with a review with the supervisor and a retrospective, and the retrospective produces at most two concrete process changes so that the list of improvements stays actionable.

Work is organised by functional module rather than by technical layer. An item such as dual contract signing therefore includes its schema change, its API endpoint, its permission entry, its interface, and its tests, and it is only counted as done when all of those parts exist. This keeps the increment demonstrable and prevents the common failure mode in which the backend is "finished" for three sprints while the interface that exercises it does not exist. The Definition of Done used for every item is stated in Section 2.1.

Communication is continuous rather than ceremonial. A short daily stand-up covers progress, the next item, and blockers; blockers that affect another member are resolved immediately after the stand-up rather than carried to the next day. Source code, the task board, and the sprint burndown live in the same GitHub repository that holds the code, so a review comment and the change it refers to are never more than one click apart. Decision records, meeting minutes, and design notes are kept in the shared documentation folder referenced in Section 6.

Risk is reviewed at every Sprint Planning rather than only at the start of the project. The register in Section 1.3 is re-scored when a new integration is introduced, and any risk whose possibility increases is paired with a concrete action in the sprint backlog instead of being left as a note. This is what turns the register into a management instrument rather than a document produced once for the report.

By applying Scrum with a fixed effort envelope, a module-oriented breakdown, and an explicit Definition of Done, the team keeps the product in a releasable state at the end of every sprint and can trade scope, rather than quality or schedule, when reality diverges from the plan.

2.1 Project Process

The Frevia project is delivered over sixteen weeks in five sprints. Sprint 1 runs for four weeks and Sprints 2 to 5 for three weeks each. The first sprint is longer because it includes the setup work that has no user-visible output but determines whether the remaining sprints can move quickly: repository structure, the shared validation package, the Prisma schema and its first migration, the Docker Compose environment, the authentication flow, and the permission guard.

The following Scrum events are held in every sprint:

- **Sprint Planning (first working day):** Confirms the sprint goal, selects items from the Product Backlog up to the sprint's effort cap, and decomposes each item into tasks of at most one day. An item enters the sprint only if its dependencies are already merged.
- **Daily Stand-up (15 minutes):** Each member states what was completed since the previous stand-up, what will be completed before the next one, and any blocker. A blocker that prevents another member from working is resolved immediately after the meeting.
- **Backlog Refinement (once per week):** Re-estimates the next sprint's candidate items and splits any item that is still too large to finish inside a sprint.
- **Sprint Review (last working day):** Demonstrates the increment on a running environment rather than through slides. The supervisor and invited users exercise the flows, and any item that cannot be demonstrated is returned to the backlog.

- **Sprint Retrospective (after the review):** Identifies what worked, what did not, and at most two process changes to apply in the next sprint. The previous sprint's changes are reviewed first so that the team can tell whether they actually helped.

Every item is measured against a single Definition of Done before it is counted as complete. The Definition of Done is deliberately mechanical so that it does not depend on the judgement of the person who wrote the code:

- The change is merged into the integration branch through a reviewed pull request, and at least one other member has approved it.
- `pnpm lint` and `pnpm check-types` pass on the merged result, and the API unit test suite (`pnpm -filter api test`) passes for the affected modules.
- `docker compose build` and `docker compose up -d` produce a running API that reports a healthy status and a web client that becomes available once the API is healthy, and the new behaviour can be exercised from the browser.
- Database changes are delivered as a migration generated from the Prisma schema, never as a manual alteration of a shared database.
- The new endpoint is registered in the permission model with the correct module, method, and path, and an unauthorised role receives a permission error rather than a successful response.
- User-facing text exists in both English and Vietnamese message catalogues; no string is hard-coded in a component.
- At least one automated test covers the new behaviour, or the pull request states explicitly why a test is not applicable.
- Documentation that describes the changed behaviour, including the API reference and the acceptance test steps, is updated in the same pull request.

Work is organised through short-lived branches named after the use case they implement, following the pattern `uc-[id]-[action-name]`. A branch is created from the integration branch, kept to a single use case, merged by pull request, and deleted afterwards. Direct pushes to the integration branch and to the stable branch are not permitted. The stable branch is updated only at the end of a sprint, from the integration branch, after the increment has been demonstrated at the Sprint Review.

Functional dependencies are sequenced explicitly rather than discovered during implementation. Identity and the permission model land before any protected module; jobs land

before proposals; proposals land before contracts; contracts land before milestones; milestones land before funding, release, and disputes. This ordering is why the contract item can assume that a proposal already exists and why the dispute item can assume that a funded milestone already exists, and it is the main reason the estimation table can be read as a dependency order from top to bottom.

Testing activity is distributed across the sprint rather than concentrated at its end. Unit and repository tests are written with the item, integration tests are extended when a state machine changes, and the end-to-end path is executed on the staging environment before the Sprint Review. An item that fails its acceptance criteria is not demonstrated, and the failure is recorded as a defect in the backlog rather than as an unfinished task, so that the sprint velocity measurement reflects delivered work only.

By applying Scrum with an explicit Definition of Done, a strict dependency order, and testing embedded in each item, the Frevia project keeps the increment demonstrable at every sprint boundary and makes the cost of any change visible before it is absorbed.

2.2 Quality Management

Quality management in Frevia is built into the delivery process rather than added as a testing phase at the end. The objective is a platform whose behaviour is predictable under failure, whose financial records can be reconciled against Stripe at any moment, and whose authorisation model cannot be bypassed by calling an endpoint directly.

- **Defect Prevention and Coding Standards**
  - Every requirement is traceable to a SMAR identifier in the estimation table, so a change can be checked against a specific item rather than against a general description.
  - Request and response shapes are declared once as Zod schemas in the shared package and consumed by both the API and the web client, which removes the class of defect in which the browser accepts an input that the server rejects, or the reverse.
  - Coding conventions are written down and enforced by ESLint 9 and Prettier: dotted lower-case file names, PascalCase classes and DTOs, camelCase functions, UPPER_SNAKE constants, no use of `any`, and no credential or secret written into source code.
  - Configuration is supplied through environment variables validated at start-up by a Zod schema, so a missing or malformed variable stops the application instead of failing later at the first request that needs it.
- **Code Review and Architecture Review**

- Every change reaches the integration branch through a pull request with at least one approving reviewer; the reviewer checks behaviour, error handling, and the permission entry rather than formatting alone.
- Controllers are restricted to request handling and response shaping; business rules live in services and database access lives in repositories. A pull request that puts a query in a controller is returned.
- Database migrations are reviewed as artefacts. A migration must be generated from the Prisma schema, must be reversible in principle, and must not drop or rename a populated column without an accompanying data step.
- Architecture decisions that affect more than one module – the permission model, the contract and milestone state machines, the dispute lifecycle, and the Stripe event handling – are reviewed with the whole team before implementation starts.

- **Software Testing**
  - Unit tests cover services, repositories, and guards with Jest. The suite already exercises the access-token guard, the authentication role transitions, the contract and proposal repositories, the payment service, the dispute service, the profile revision workflow, the curriculum vitae analyser, and the profile strength calculation.
  - Integration tests run against a real PostgreSQL database configured through environment variables, so a query that is syntactically valid but semantically wrong is caught before the sprint review.
  - An end-to-end test drives the application through its HTTP surface and verifies the response envelope, the validation behaviour, and the authorisation behaviour of protected routes.
  - Manual exploratory testing is performed on the flows that are difficult to automate reliably: the identity document capture sequence, the Stripe Checkout redirect, the dispute evidence upload, and the real-time messaging session.
  - Each item is accepted against written acceptance criteria at the Sprint Review, and an item that fails is recorded as a defect rather than counted as delivered.

- **Security and Data Protection**
  - Every route is protected by the permission guard unless it is explicitly marked public, and the guard resolves the permission from the role that is active in the session rather than from a role name embedded in the token.
  - Passwords are stored as bcrypt hashes; refresh tokens are persisted per session so that a session can be revoked individually; one-time codes are stored hashed with an attempt counter and an expiry.

  - Identity documents are served only through authenticated endpoints that check ownership or administrator rights, and are never exposed through the public static upload directory.
  - Input is validated at the boundary with the shared schemas, Prisma parameterises every query, and cross-origin access is restricted to the configured web origin.
  - Access control is checked negatively as well as positively: every protected endpoint is exercised with the wrong role during review, and the access-token guard is covered by automated tests.

- **Transaction and Payment Validation**
  - All payment work is performed in Stripe test mode. The webhook endpoint verifies the signature against the raw request body before parsing the event.
  - Handlers are idempotent: the same event delivered twice must not create a second transaction, release a milestone twice, or duplicate a dispute fee.
  - Local state is reconciled from Stripe before it is written, and explicit synchronisation endpoints exist for PaymentIntents and Checkout Sessions so that a missed webhook can be repaired without a database edit.
  - Every money movement is written to the transaction ledger with a type, a status, and the corresponding Stripe identifier, which makes a contract's financial history reconstructable and auditable.

- **Performance and Quality Metrics**
  - List endpoints are paginated and filtered in the database rather than in memory, and the queries used by the job list, proposal list, moderation queue, and notification list are checked against their indexes.
  - The team tracks test results, defect counts by severity, item completion against the sprint cap, and the response time of the heaviest endpoints.
  - Defects are classified as critical, high, medium, or low, and the resolution targets in Section 1.2 are applied to that classification.
  - Additional performance checks are performed before the acceptance milestone on the messaging gateway and on the job search endpoint, which are the two paths most sensitive to load.

- **Scrum-Based Quality Improvement**
  - Quality activities are planned as part of each item rather than as a separate task at the end of the sprint.

- Acceptance criteria are written during Sprint Planning, before implementation, so that the reviewer and the developer agree on what "done" means.
- Defects found during the sprint are fixed inside the same sprint when they block a committed item, and are otherwise returned to the backlog with a severity.
- Each retrospective produces at most two process changes, and the following retrospective begins by checking whether they were applied.

- **Feedback and Issue Management**
  - Defects, technical debt, and enhancement requests are recorded as issues in the same repository as the code, with a label, a severity, and the SMAR identifier they affect.
  - Feedback from the supervisor and from acceptance test participants is triaged at Sprint Planning rather than acted on immediately, so that the current sprint remains stable.
  - A defect that reveals a gap in the requirements produces a documentation change as well as a code change, so that the same gap is not reintroduced later.
  - The issue list and the test results are reviewed together at the end of each sprint to decide whether the increment is releasable.

These practices target the failure modes that matter most in this system: an authorisation bypass, an inconsistent payment record, a lost identity document, and a defect that only appears when two modules interact. Integrating review, testing, and feedback into each sprint keeps the product releasable at every sprint boundary instead of concentrating risk at the end of the project.

2.3 Training Plan

Training is scheduled before the sprint in which the related technology is first used, so that no member learns a technology while also carrying a committed item that depends on it. Each session is delivered by the member who owns the corresponding area, lasts the stated duration, and ends with a small exercise that is committed to the repository.

Training Area	Participants	When, Duration	Waiver Criteria
Git workflow, branch naming, and pull request review	All members	Week 1 (2 days)	Mandatory

Continued on next page

Training Area	Participants	When, Duration	Waiver Criteria
Monorepo tooling: pnpm workspaces, Turborepo, shared TypeScript configuration	All members	Week 1 (2 days)	Mandatory
Next.js App Router, React 19, and server versus client components	Frontend members	Week 1 (1 week)	Mandatory for frontend work
NestJS modules, dependency injection, guards, and interceptors	Backend members	Week 1–2 (1 week)	Mandatory for backend work
Prisma schema design, migrations, and seeding	All members	Week 2 (3 days)	Mandatory
PostgreSQL modelling for transactional workflows and indexing	Backend members	Week 2 (3 days)	Mandatory for backend work
Shared Zod schemas and the permission-based authorisation model	All members	Week 3 (2 days)	Mandatory
Tailwind CSS 4, shared component library, and bilingual message catalogues	Frontend members	Week 3 (3 days)	Mandatory for frontend work
Docker Compose, environment configuration, and secret management	All members	Week 4 (2 days)	Mandatory
Stripe Connect onboarding, PaymentIntent, Checkout Session, and webhook verification	Backend members and payment owner	Week 4 (1 week)	Mandatory before Sprint 4
Socket.IO gateway, JWT handshake authentication, and room-based delivery	Messaging owners	Week 5 (3 days)	Mandatory before the messaging items

Continued on next page

Training Area	Participants	When, Duration	Waiver Criteria
External AI service integration: curriculum vitae analysis and the identity verification pipeline	Verification owners	Week 6 (3 days)	Mandatory before Sprint 5
UI/UX design with Figma and design hand-off	All members	Project start (3 days)	Recommended

The Stripe session is placed at the end of Sprint 1 rather than at the start of Sprint 4 because payment work touches the contract, milestone, and dispute modules, and the team needs to see those modules before the payment flow makes sense. The same reasoning applies to the Socket.IO session, which follows the contract module, and to the AI integration session, which follows the milestone module because identity verification is attached to the freelancer profile that the milestone flow already exercises.

3. Project Deliverables

The project produces both internal deliverables, such as plans, specifications, and test artefacts, and external deliverables, such as the running increments and the final report. The table below lists them in the order in which they are due.

#	Deliverable	Due Date	Notes
1	Product Introduction Document	Week 1	Sprint 1, 7 days
2	Project Management Plan	Week 2	Sprint 1, 7 days
3	Software Requirements Specification	Week 3	Sprint 1, 8 days
4	Software Design Specification	Week 4	Sprint 1, 8 days
5	Increment 1: identity and access, profiles, job marketplace, forum foundation	Week 4	Sprint 1, 18 days
6	Sprint 1 review and retrospective	Week 4	Sprint 1, 1 day
7	Increment 2: proposals, contracts, milestones and submissions	Week 7	Sprint 2, 14 days
8	Updated SRS and SDS (iteration 2)	Week 6	Sprint 2, 2 days
9	Test cases for proposal, contract and milestone flows	Week 7	Sprint 2, 3 days
10	Sprint 2 review and retrospective	Week 7	Sprint 2, 1 day

#	Deliverable	Due Date	Notes
11	Increment 3: real-time messaging, notifications, administration and moderation	Week 10	Sprint 3, 14 days
12	Updated SRS and SDS (iteration 3)	Week 9	Sprint 3, 2 days
13	Test cases for messaging, moderation and identity review	Week 10	Sprint 3, 3 days
14	Sprint 3 review and retrospective	Week 10	Sprint 3, 1 day
15	Increment 4: Stripe escrow, disputes, banners and catalogues	Week 13	Sprint 4, 14 days
16	Updated SRS and SDS (iteration 4)	Week 12	Sprint 4, 2 days
17	Stripe test-mode integration suite and webhook scenario tests	Week 13	Sprint 4, 3 days
18	Sprint 4 review and retrospective	Week 13	Sprint 4, 1 day
19	Increment 5: AI curriculum vitae analysis, expert directory, social features and localisation	Week 16	Sprint 5, 14 days
20	Quality hardening and full regression run	Week 15	Sprint 5, 4 days
21	User acceptance testing and defect fixing	Week 16	Sprint 5, 5 days
22	Deployment package and environment guide	Week 16	Sprint 5, 2 days
23	Final project report	Week 16	Sprint 5, 5 days
24	Thesis presentation and live demonstration	Week 16	Sprint 5, 3 days
25	Updated SRS and SDS (final revision)	Week 16	Sprint 5, 2 days

Table 6: Project Deliverables

The increments are cumulative. Increment 2 assumes that jobs and profiles from Increment 1 already exist, Increment 3 assumes that contracts and milestones already exist, Increment 4 assumes that a funded milestone already exists, and Increment 5 assumes that the freelancer profile and the verification record already exist. Document deliverables are produced once and then revised at each iteration instead of being rewritten, which is why the SRS and the SDS appear four times in the table.

4. Responsibility Assignments

This section defines who does what in the Frevia project. It first describes the structure of the team and its interfaces with the supervisor and the users, then lists the roles that the members take on, and finally assigns those roles to each of the project's outputs through a responsibility matrix.

4.1 Team & Structure

Frevia is delivered by a single cross-functional team of five members rather than by separate frontend, backend, and testing groups. Each member holds a primary role, listed in Section 4.2, and takes one or more secondary roles for the duration of a sprint. The team leader acts as Product Owner and is accountable for the backlog order and for accepting each increment; the same member facilitates the Scrum events, so that the plan and the delivery cannot drift apart without being noticed.

The structure is deliberately flat. There is no separate quality department: the member who implements an item is also responsible for its unit tests, and the reviewer of the pull request is responsible for confirming that the Definition of Done in Section 2.1 was respected. Test ownership rotates between sprints so that each member has reviewed at least one module they did not write before the acceptance milestone.

Two interfaces sit outside the team. The project supervisor reviews the increment at the end of every sprint, provides feedback on the scope and on the documentation, and approves the submission of each report. Prospective users participate in acceptance testing in the final sprint, but they do not join sprint planning and do not negotiate scope directly with the team; their feedback reaches the backlog through the Product Owner.

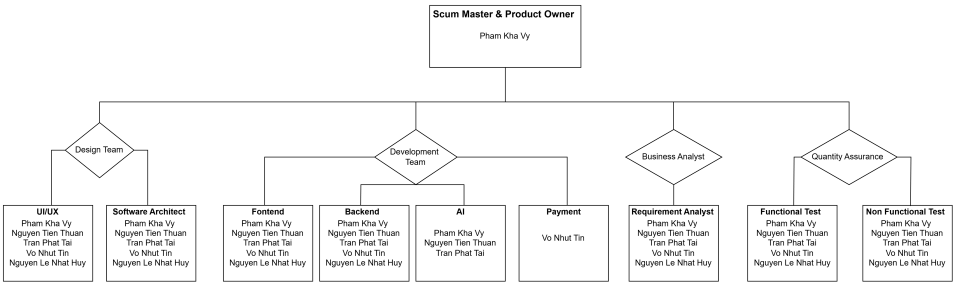


Figure 1: Team and Structure Diagram

4.2 Role & Responsibilities

Role Name	Description
Product Owner	Owns the product direction, orders the backlog by business value and dependency, and accepts or rejects each increment at the Sprint Review. In this project the role is combined with the team leader.
Scrum Master	Facilitates the Scrum events, protects the sprint commitment from unplanned work, tracks the burndown, and removes blockers that the team cannot resolve on its own.

Role Name	Description
UX/UI Designer	Defines the user flows for the client, freelancer, expert, and administrator journeys, produces the interface designs, and maintains the shared component library used by the web client.
Software Architect	Defines the module boundaries of the monorepo, the data model, the contract and milestone state machines, the permission model, and the standards that a change must satisfy before it is merged.
Frontend Developer	Implements the Next.js application: routing, forms, data fetching, bilingual message catalogues, and the interface states for loading, empty, error, and permission-denied conditions.
Backend Developer	Implements the NestJS modules: controllers, services, repositories, validation schemas, guards, and the database migrations that support them.
AI Integration Developer	Integrates the two external AI services that the platform depends on – curriculum vitae analysis and identity document verification. Defines the request contract, the timeout and error behaviour, the validation of returned data, and the manual fallback used by administrators when a service is unavailable. No model is trained in this project.
Payment Integration Developer	Owns the Stripe integration: Connect Express onboarding, platform fee and milestone funding through PaymentIntent and Checkout Session, transfers on release, refunds, dispute fees, settlement, webhook verification, and reconciliation of the transaction ledger against Stripe.
Requirement Analyst	Bridges the stakeholders and the development team: elicits requirements, writes the use cases and acceptance criteria, and keeps the requirement specification consistent with what has actually been delivered.
Functional Tester	Verifies each delivered item against its acceptance criteria, writes and maintains the test cases, reproduces reported defects, and confirms fixes before an item is demonstrated at the Sprint Review.
Non-Functional Tester	Evaluates quality attributes rather than features: authorisation bypass attempts, input validation, response times on list endpoints, behaviour under concurrent messaging sessions, and the stability of the deployment.

Role Name	Description
Release and Environment Engineer	Maintains the containerised development environment, the environment variable contract, the secret provisioning through Infisical, the migration procedure, and the deployment checklist used at the end of each sprint.

Table 7: Roles and Responsibilities

4.3 Responsibility Assignment Matrix

The matrix assigns every output of the project to the members of the team. It uses the notation of the report template:

D Do; R Review; S Support; I Informed; <blank> - Omitted

Responsibility	VyPK	ThuanNT	TaiTP	TinVN	HuyNLN
Project initiation, scope and risk planning	DR	SI	SI	SI	SI
Product Introduction Document	DR	SI	SI	SI	SI
Project Management Plan	DR	SI	SI	SI	SI
Requirements gathering and user stories	R	D	D	SI	SI
UI/UX prototype and component library	R	SI	D	SI	D
Software architecture and data model	R	D	D	D	D
Authentication, permissions and session management	R	D	SI	SI	S
Profile, skills, portfolio and curriculum vitae handling	R	S	SI	D	D
Job marketplace, bookmarks and saved search	R	S	D	SI	D
Proposal and hiring workflow	R	S	D	SI	S
Contract signing, milestones and submissions	R	D	S	SI	D
Stripe escrow, release, refund and reconciliation	R	S	SI	D	SI

Responsibility	VyPK	ThuanNT	TaiTP	TinVN	HuyNLN
Dispute arbitration and settlement	R	D	S	S	SI
Real-time messaging and notifications	R	D	SI	S	D
Forum, moderation and content administration	DR	SI	S	SI	D
AI integration: identity verification and CV analysis	R	S	SI	D	SI
Administrative console, catalogues and banners	DR	S	SI	SI	D
Frontend development	R	SI	D	SI	D
Backend development	R	D	D	D	SI
Unit testing	R	D	S	D	S
Integration and end-to-end testing	R	S	D	D	D
Payment and webhook security testing	R	S	SI	D	SI
User acceptance testing	DR	I	I	I	I
Technical training	DR	D	D	D	D
Deployment and environment configuration	R	D	SI	D	SI
Final project report	DR	D	D	D	D
Thesis presentation	DR	D	D	D	D

Table 8: Responsibility Assignments Matrix

Every row has exactly one member who reviews the work: that member confirms the output before it is demonstrated at the Sprint Review and reports its status. Where a row lists several members who do the work, the work is split by module rather than duplicated, and the split is recorded on the task board before the sprint begins. The rows that carry the highest technical risk – Stripe escrow and reconciliation, dispute arbitration, and AI integration – pair the member who does the work with at least one supporter so that a second person can check the implementation.

5. Project Communications

Effective communication depends on a fixed cadence and on a single place where decisions are recorded. Every recurring activity below produces an artefact – a meeting note, a reviewed pull request, an issue, or an updated document – so that a decision made in a conversation does not have to be remembered by the people who attended it.

Communication Item	**Who/ Target**	**Purpose**	**When, Frequency**	**Type, Tool, Method(s)**
Daily stand-up	Development team	Report progress since the previous stand-up, state the next item, and raise blockers	Daily, 15 minutes	Discord voice channel, short meeting note
Backlog refinement	Development team	Re-estimate candidate items, split oversized items, and confirm dependencies	Weekly	GitHub project board, Discord discussion
Sprint Planning	Team members, supervisor	Confirm the sprint goal and the commitment up to the effort cap	Start of each sprint	Google Meet, GitHub issues
Sprint Review	Team members, supervisor, invited users	Demonstrate the increment on a running environment and collect feedback	End of each sprint	Google Meet, live demonstration on the staging environment
Sprint Retrospective	Development team	Agree at most two process changes and verify the previous sprint's changes	End of each sprint	Discord, retrospective note in the documentation folder
Design and schema review	Team members, module owner	Agree on the data model, the API contract, and the permission entry before implementation	Before each high-complexity item	Google Meet, design note committed to the repository
Pull request review	Author and at least one reviewer	Check behaviour, error handling, authorisation, and test coverage	Per change	GitHub pull request review
Defect triage	Team members	Classify defects by severity and assign an owner	Twice per sprint	GitHub issues with severity labels
Weekly progress report	Team leader, supervisor	Report status against the sprint cap, the risk register, and any support required	Weekly	Written report sent by email, Google Meet when a discussion is needed

Table 9: Project Communication Plan

Communication is conducted through Google Meet for synchronous reviews and supervisor meetings, Discord for day-to-day coordination and quick technical discussion, GitHub for code, pull request discussion, and the issue list, and Google Drive for documents, reports, and shared figures. Figma is used for interface design hand-off, where comments on a frame are resolved before the corresponding frontend item is started.

Meeting outcomes are recorded in a short note with the date, the participants, the decisions taken, and the action items with an owner. The note is stored in the shared documentation folder and linked from the task board, so a member who missed the meeting can recover the decision without asking. A decision that changes scope, schedule, or an existing interface is also recorded in the relevant document rather than only in the meeting note, and the change is announced in the next stand-up.

6. Configuration Management

6.1 Document Management

The project documents are managed as versioned artefacts under the `docs` directory of the same repository that holds the source code. Keeping the documents next to the code means that a change to behaviour and the change to the document that describes it can be reviewed in the same pull request, which is the only reliable way to prevent the specification from drifting away from the delivered system.

The managed documents include:

- Product Introduction (Report 1), describing the background, existing systems, business opportunity, product vision, and the feature and limitation lists
- Project Management Plan (Report 2), describing scope and estimation, objectives, risks, organisation, deliverables, communication, and configuration management
- Software Requirements Specification (Report 3), containing the actors, use cases, and sprint mapping
- Software Design Specification, containing the module structure, data model, and state machines
- Test cases and test reports for each iteration
- User guide and deployment guide
- Meeting minutes and decision notes

Documents are written in LaTeX and stored as source files with their figures in a local `image` directory, so that a document can be rebuilt and diffed rather than being edited as a

binary file. The PDF submitted at the end of an iteration is exported from the same source, and the export is repeated whenever the source changes.

Versions follow a two-level scheme: the minor number increases for a review draft or a correction that does not change scope, and the major number increases when a baseline is approved and the document is submitted. A document therefore moves from 0.1 to 0.2 while it is being reviewed, and to 1.0 when it is submitted, and the version and the date appear on the cover page of every submission.

Change control is applied to the Project Management Plan only. A change to the scope, the sprint allocation, the effort estimate, or the deliverables requires an update to the plan together with a short note describing what changed and why. Documents that describe the product rather than the plan are revised freely, because their purpose is to remain accurate rather than to remain stable.

Obsolete versions are retained rather than deleted so that a reviewer can reconstruct what was agreed at a given submission. Each archived version keeps an explicit version suffix in its file name, so that two exports of the same document can never be confused.

6.2 Source Code Management

Source code is managed in a single monorepo versioned with Git and hosted on GitHub. The repository contains the API application, the web application, the shared packages, the database schema, and the documentation, which allows a change that spans the frontend, the backend, and a shared validation schema to be reviewed and merged atomically.

The following branching strategy is applied:

- `main`: the stable branch. It is updated only at the end of a sprint, from `develop`, after the increment has been demonstrated at the Sprint Review.
- `develop`: the integration branch. Every member integrates into this branch, and it is expected to build and pass the test suite at all times.
- `uc-[id]-[action-name]`: short-lived feature branches created from `develop` for a single use case, for example `uc-09-sign-contract`.
- `hotfix/[short-description]`: urgent corrections created from `main`, merged back into both `main` and `develop` so that the fix is not lost at the next integration.

A change is merged only through a pull request. The pull request must reference the SMAR identifier it implements, describe the behaviour that changed, and state how it was verified. At least one other member must approve it, and the author may not approve their own change. The project's Definition of Done is applied at review time: the lint, type-check, and test

commands pass, the container build succeeds, the new endpoint is registered in the permission model, and the user-facing text exists in both language catalogues.

Commit messages are written in the imperative mood with a short scoped summary, so that the history of a module can be read without opening the diff. A commit that changes the database schema is separated from the commit that changes application behaviour, which makes a migration easy to identify and to revert independently.

Direct pushes to `develop` and to `main` are not permitted, and neither branch may receive a force push after it has been shared. Branches are deleted after merge so that the branch list reflects only work that is in progress.

Generated artefacts are excluded from version control: build output, dependency directories, uploaded files, coverage reports, and environment files. Secrets are never committed; configuration is supplied through environment variables loaded from the secret manager described in Section 6.3, and the committed environment template contains only the names of the variables and placeholder values.

The repository layout mirrors the architecture: `apps/api` contains the NestJS application with its Prisma schema and migrations, `apps/web` contains the Next.js application, `packages/shared-type` contains the Zod schemas and the types derived from them, `packages/ui` contains the shared interface components, and `docs` contains the reports and the specifications. The deployment definition, the workspace configuration, and the task pipeline live at the repository root so that a fresh clone can be started with a single command.

6.3 Tools & Infrastructures

Category	**Tools / Infrastructure**	**Technology**
Project tracking	Issue list, milestones, and the project board kept in the same repository as the code	GitHub
Version control	Two long-lived branches and short-lived use-case branches merged by pull request	Git, GitHub
Communication	Daily coordination, sprint reviews, and supervisor meetings	Discord, Google Meet
Documentation	LaTeX sources in the repository, shared material, and required forms	LaTeX, Google Drive, Microsoft Word
UI/UX design	User flows, wireframes, and the shared component library	Figma

Category	Tools / Infrastructure	Technology
Runtime and workspace	Workspace management and task orchestration for both applications	Node.js, pnpm, Turborepo
Frontend	Next.js App Router application with the shared component library, server state, and the forum editor	Next.js 16, React 19, Tailwind CSS 4, next-intl, TanStack Query, Tiptap
Backend	NestJS API with a global prefix, request validation, a permission guard, and the messaging gateway	NestJS 11, nestjs-zod, Socket.IO
Data access	Client and migration tool generating every schema change from the data model	Prisma 7
Database	Relational database supporting the transactional workflows and their indexes	PostgreSQL
Shared contracts	Shared package containing the validation schemas and the types used by both applications	TypeScript, Zod
Payments	Escrow funding, release, refunds, dispute settlement, and a signature-verified webhook, all in test mode	Stripe (Connect Express, PaymentIntent, Checkout Session)
Media storage	Profile and portfolio images; identity documents stored on the application's own storage	Cloudinary
Email	One-time verification and password reset codes	SMTP, Nodemailer
External AI services	Curriculum vitae analysis and identity verification, called only from the backend over HTTP with an explicit timeout	External AI APIs
Testing	Unit, integration, and end-to-end tests, plus exploratory API checks	Jest 30, Postman
Quality tooling	Linting, formatting, and strict type checking	ESLint 9, Prettier, TypeScript strict mode

Category	Tools / Infrastructure	Technology
Secret management	Environment variables injected at start-up; no secret stored in the repository	Infisical
Containerisation	API and web client containers, with a health check before the client starts	Docker, Docker Compose
API documentation	Maintained reference describing each endpoint, its permission, and its request and response schemas	Markdown in the repository

Table 10: Tools and Infrastructures

The tool selection follows a single constraint: a member must be able to reproduce the whole system from a fresh clone with one command. That is why the runtime, the database, the task pipeline, and the container definition are all pinned in the repository rather than installed manually per machine. It is also why the configuration is injected through a secret manager instead of being copied between members, and why the API reference is kept in the repository rather than in a separately hosted portal.

Two capabilities are deliberately not part of the toolchain. There is no automated continuous integration service: the same quality gate is executed locally and inside the container build, and the reviewer confirms the result before approving a pull request. There is no hosted API documentation portal; the reference is a maintained document, which is sufficient for a team of five and avoids a second source of truth that can fall out of date.